МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

Інститут комп’ютерних технологій, автоматики та метрології

Кафедра "Комп'ютеризовані системи автоматики"



ЗВІТ

про виконання **курсової роботи**

“Розробка Розумної Парковки”

з навчальної дисципліни: **“Мікроконтролери, частина 1”**

Виконав:

студент групи ІР-24

Яремчук Станіслав

Прийняв:

Татарин В. Я.

Львів - 2026

1. МЕТА ТА ЗАВДАННЯ КУРСОВОГО ПРОЕКТУ

Метою курсового проекту є проектування та розробка комплексної автоматизованої системи керування паркувальним простором (Smart Parking) на базі мікроконтролера ESP32. Проект спрямований на інтеграцію апаратних засобів моніторингу об'єктів у реальному часі з сучасними веб-технологіями для збору, зберігання та візуалізації даних.

Для досягнення поставленої мети необхідно вирішити наступні завдання:

1.1. Проектування апаратної частини:

- Проектування та збірка схеми на базі мікроконтролера ESP32, що виступає центральним вузлом обробки даних.
- Інтеграція ультразвукових сенсорів HC-SR04 для точної детекції наявності автомобілів на паркувальних місцях та перед в'їздом.
- Реалізація системи автоматичного керування шлагбаумом за допомогою сервоприводу.
- Впровадження модуля метеомоніторингу на основі датчика DHT22 для збору показників температури та вологості.
- Забезпечення візуальної індикації станів (вільно/зайнято) за допомогою двоколірних світлодіодів.
- Здійснити монтаж компонентів системи шляхом створення надійних паяних з’єднань.

1.2. Проектування та розробка бази даних:

- Створення реляційної моделі даних у MySQL для збереження поточного стану об'єктів та історії подій.
- Розробка структури таблиць для логування заїздів/виїздів та накопичення погодинної статистики метеоданих.

1.3. Створення серверної частини (Back-end):

- Розробка RESTful API на мові програмування Python з використанням фреймворку Flask для обробки запитів від мікроконтролера та веб-інтерфейсу.

1.4. Розробка веб-інтерфейсу (Front-end):

- Створити інтерактивний Dashboard (HTML/CSS/JavaScript) для візуалізації завантаженості парковки в реальному часі.
- Реалізувати можливість дистанційного бронювання вільних місць через інтерфейс браузера.

1.5. Тестування та демонстрація:

- Перевірка працездатності системи на фізичному обладнанні.
- Підготовка демонстраційних матеріалів, що підтверджують коректну взаємодію всіх ланок системи: від фізичного датчика до оновлення даних у веб-інтерфейсі.

2. ВСТУП

Сьогодні кожен, хто приїжджає на власному авто до великого торгового центру в суботу ввечері або поспішає на рейс в аеропорт, стикається з однією і тією ж проблемою - де припаркуватися? Величезні парковки часто переповнені, і водії змушені кружляти десятками хвилин у пошуках вільного місця. Це не тільки дратує, а й створює зайві затори та загазовує територію.

Навіть якщо на в'їзді в аеропорт стоїть табличка "Місць немає", ми не знаємо, коли саме воно звільниться. Вирішенням цієї проблеми є "розумні" системи, які автоматично стежать за кожним метром парковки. Саме таку систему я і розробив у своєму курсовому проекті.

Мій проект - це інтелектуальна система «Smart Parking» на базі мікроконтролера ESP32. Вона працює просто і надійно: датчики бачать машину на місці, передають цю інформацію через Wi-Fi на сервер, а водій може зайти на сайт зі свого телефону і відразу побачити, куди йому їхати.

Особливість моєї розробки в тому, що я пройшов повний шлях створення реального продукту:

- власноруч спаяв усі модулі та з'єднав датчики;
- написав програму, яка керує шлагбаумом та світлодіодами;
- створив базу даних та зручний сайт для моніторингу.

Така система допоможе власникам ТЦ краще керувати парковкою, а водіям — економити час та нерви, точно знаючи, що на них чекає вільне місце.

3. АНАЛІЗ ПИТАННЯ

У даному розділі проведено огляд та аналіз технологій, обраних для реалізації системи розумного паркування. Вибір інструментів зумовлений необхідністю забезпечення стабільного зв’язку між апаратною частиною та сервером, швидкої обробки даних та наочної візуалізації результатів

3.1. Апаратна платформа: ESP32

Вибір мікроконтролера ESP32 обумовлений його високою продуктивністю та наявністю вбудованого стеку Wi-Fi/Bluetooth, що є критично важливим для IoT-пристроїв. На відміну від платформи Arduino, ESP32 працює на вищій тактовій частоті та має більший обсяг оперативної пам'яті, що дозволяє без затримок обробляти JSON-пакети та підтримувати шифровані з'єднання з сервером.

3.2. Сенсори та виконавчі механізми

У проекті використано наступну периферію:

- **Ультразвуковий далекомір HC-SR04:** Обраний як основний засіб детекції через свою стійкість до умов освітлення (на відміну від інфрачервоних датчиків). Він забезпечує точність вимірювання у діапазоні від 2 см до 4 м.
- **Датчик кліматичного моніторингу DHT22:** Використаний для зчитування показників температури та вологості повітря. Він має

цифровий інтерфейс передачі даних та вищу точність порівняно з бюджетною моделлю DHT11.

- **Сервопривід (Micro Servo SG90):** Застосований для фізичної реалізації механізму шлагбаума.

Electric Parameter

Working Voltage	**DC 5 V**
Working Current	**15mA**
Working Frequency	**40Hz**
Max Range	**4m**
Min Range	**2cm**
MeasuringAngle	**15 degree**
Trigger Input Signal	**10uS TTL pulse**
Echo Output Signal	**Input TTL lever signal and the range in proportion**
Dimension	**45*20*15mm**

Рис. 3.1 Datasheet HC-SR04(сайт)

3.3. Програмне середовище та мови програмування

- **C++ (Arduino IDE):** Використана для написання прошивки мікроконтролера. Це дозволило ефективно керувати перериваннями для датчиків HC-SR04 та реалізувати логіку взаємодії з HTTP-клієнтом.
- **Python (Flask):** Обраний для розробки Back-end частини завдяки високій швидкості розробки RESTful API. Фреймворк Flask забезпечує стабільне прийняття POST-запитів від ESP32 та взаємодію з базою даних.
- **MySQL:** Обрана як система керування базами даних через свою надійність та простоту інтеграції з Python-скриптами. Вона дозволяє організувати структуроване зберігання логів паркувальних сесій та метеоданих.

3.4. Протоколи передачі даних

Взаємодія між ESP32 та сервером базується на протоколі HTTP, а дані передаються у форматі JSON. Це забезпечує легкість парсингу інформації як на стороні сервера (Python), так і на стороні клієнта (JavaScript).

4. СТРУКТУРНА ТА ФУНКЦІОНАЛЬНА СХЕМА ПРОЕКТУ

Для розуміння принципів роботи системи «Smart Parking» було розроблено структурну схему, яка відображає взаємодію між апаратними компонентами, серверною частиною та інтерфейсом користувача.

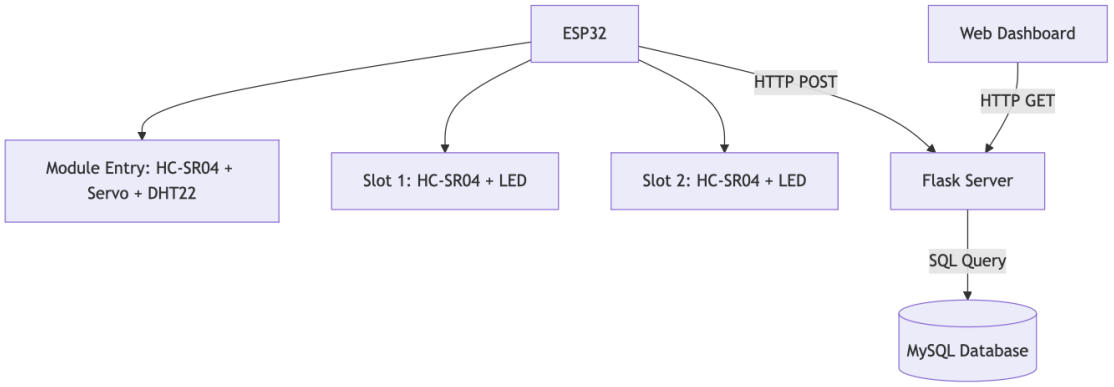


Рис. 4.1 Структурна схема взаємодії компонентів системи «Smart Parking»

4.1. Опис структурних рівнів системи

Згідно зі схемою, систему можна розділити на три основні рівні:

1. **Апаратний рівень (Hardware Layer):**
    - Центральним пристроєм є мікроконтролер ESP32, який виконує роль шлюзу.

    - До нього підключені периферійні пристрої: три ультразвукові датчики HC-SR04 (для в’їзду та двох паркомісць), датчик клімату DHT22, сервопривід шлагбаума та світлодіодна індикація.
    - Цей рівень відповідає за фізичне зчитування даних та виконання локальних команд (відкриття шлагбаума, зміна кольору LED).
2. **Мережевий та серверний рівень (Cloud/Server Layer):**
    - Передача даних здійснюється через протокол HTTP у форматі **JSON** за допомогою Wi-Fi з’єднання.
    - **Flask Server (Back-end):** приймає запити від ESP32, обробляє їх та керує потоками інформації.
    - **MySQL Database:** забезпечує надійне зберігання статусів парковки та накопичення статистики.
3. **Рівень представлення (Application Layer):**
    - **Веб-додаток (Dashboard):** отримує дані з бази через API та візуалізує їх для кінцевого користувача.
    - Цей рівень також дозволяє надсилати зворотні команди (наприклад, для бронювання місця).

4.2. Функціональний алгоритм роботи

Функціонування системи відбувається за наступним циклом:

- **Крок 1:** Датчики HC-SR04 вимірюють відстань. Якщо відстань менша за встановлений поріг (15 см), ESP32 фіксує зміну статусу паркомісця.
- **Крок 2:** ESP32 формує повідомлення і через Wi-Fi надсилає його на Flask-сервер.
- **Крок 3:** Сервер оновлює інформацію в базі даних MySQL.
- **Крок 4:** Веб-інтерфейс, використовуючи JavaScript, кожні кілька секунд опитує сервер та оновлює стан «віртуальної парковки» на екрані користувача.

5. ДЕТАЛЬНІ ОПИСИ СКЛАДОВИХ ПРОЕКТУ

5.1. Апаратна частина та модульна структура системи

Система «Smart Parking» спроектована як багаторівневий апаратний комплекс, де кожен вузол виконує свою специфічну функцію. Основна увага при розробці була приділена надійності з'єднань та точності зчитування даних з датчиків.

5.1.1. Центральний обчислювальний модуль (ESP32)

Вибір ESP32 DevKit V1 як центрального контролера обумовлений необхідністю одночасної роботи з великою кількістю периферії та підтримкою Wi-Fi стеку.

- **Функціонал:** Контролер виконує циклічне опитування датчиків через GPIO-виводи, обробляє отримані часові інтервали від ультразвукових сенсорів і формує HTTP-запити.
- **Живлення:** Датчики та сервопривід потребують 5В, живлення організовано через шину VIN, що підключена до стабілізованого джерела живлення. Це дозволяє уникнути просадок напруги при передачі даних через Wi-Fi.

5.1.2. Модуль в’їзду та моніторингу середовища(Модуль 1)

Це найбільш складний вузол системи, який відповідає за фізичний доступ автомобілів на територію та збір кліматичних даних.

- **Склад модуля:**
    - **Ультразвуковий датчик HC-SR04:** встановлений перед шлагбаумом. Він працює як тригер: коли автомобіль наближається на відстань менше 10 см, система ініціює процедуру відкриття.
    - **Сервопривід SG90:** механічно з’єднаний із моделлю шлагбаума. Керується через ШІМ-сигнал, повертаючи важіль на кут 90° для пропуску авто.
    - **Датчик DHT22:** інтегрований у цей модуль для вимірювання температури та вологості. Він винесений на зовнішню частину

корпусу для отримання точних показників навколишнього середовища.

5.1.3. Модуль паркомісця №1 (Модуль 2)

Цей вузол призначений для контролю конкретного паркувального слоту та візуального інформування водія.

- **Склад модуля:**
    - **Ультразвуковий датчик HC-SR04 (Slot 1):** розміщений вертикально над паркомісцем або на фронтальній стінці.
    - **Світлодіодної індикації:** rgb світлодіод (червоний та зелений, синій в повітрі).
- **Функціонал:** Датчик постійно вимірює відстань до поверхні паркомісця. Якщо відстань скорочується (машина заїхала), ESP32 миттєво змінює стан світлодіода. Зелений гасне, а червоний загоряється, що сигналізує про зайнятість місця.

5.1.4. Модуль паркомісця №2 (Модуль 3)

Це ідентичний до першого паркомісця вузол, що дозволяє системі працювати в багатоканальному режимі.

- **Склад модуля:** Аналогічний до Slot 1 (HC-SR04 + LED).
- **Робота в системі:** Незважаючи на те, що апаратно він копіює перший модуль, у програмі він має власні унікальні піни (D19, D21 для датчика та D14, D27 для LED). Це дозволяє серверу ідентифікувати, яке саме місце було зайняте, та відображати це на веб-панелі.

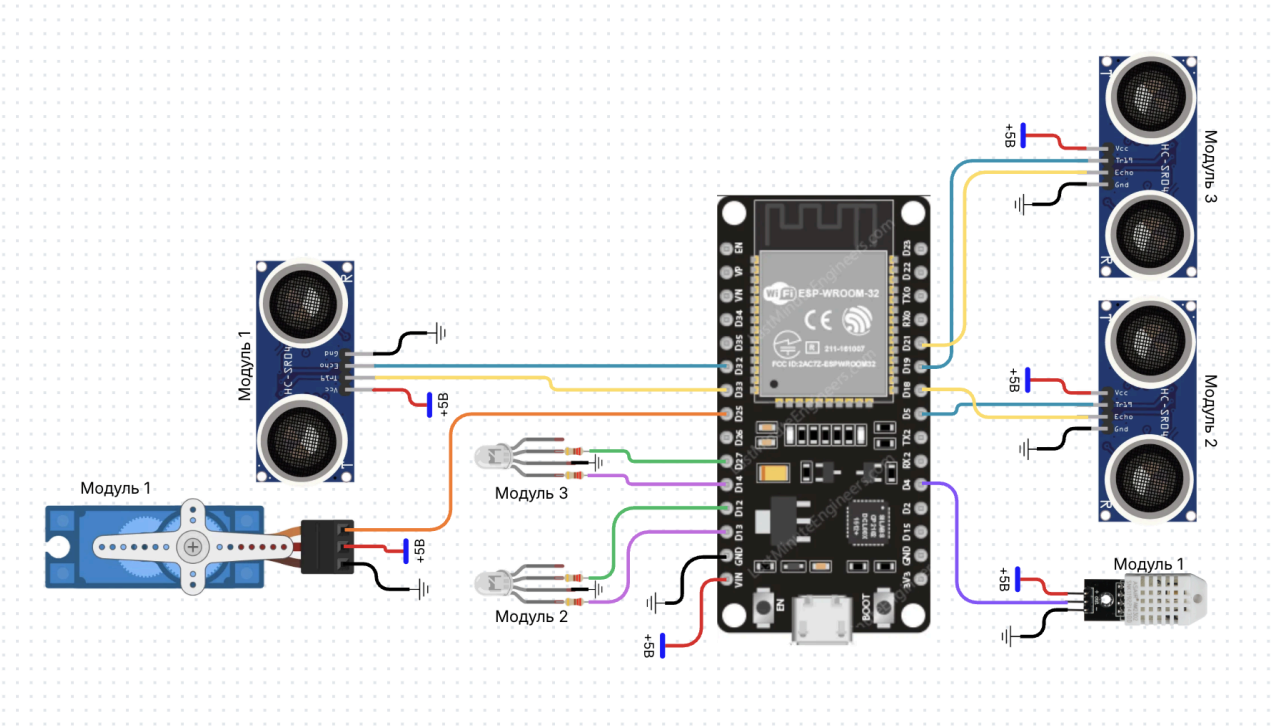


Рис. 5.1 Схема підключення компонентів.

5.2. Програмна реалізація Back-end частини

Серверна частина системи «Smart Parking» реалізована на мові Python з використанням фреймворку Flask. Вона виконує роль центрального вузла, який обробляє запити від апаратної частини (ESP32) та взаємодіє з базою даних для надання інформації користувачам.

5.2.1. Технологічний стек та конфігурація

Основою серверної частини є наступні технології:

- **Flask-SQLAlchemy:** використана як основний інструмент для роботи з базою даних. Вона дозволяє описувати структуру таблиць у вигляді Python-класів (моделей), що спрощує маніпуляцію даними.
- **CORS (Cross-Origin Resource Sharing):** налаштовано для можливості взаємодії сервера з веб-інтерфейсом, розгорнутим на іншому хості чи порті.

5.2.2. Моделі даних (Data Models)

В архітектурі сервера виділено дві ключові моделі (таблиці):

1. **ParkingSlot:** зберігає поточний стан кожного місця (ID, статус 0/1 та прапор бронювання is_reserved).
2. **HourlyStat:** призначена для накопичення статистики. Вона зберігає кількість вільних місць, а також показники температури та вологості у прив'язці до часу.

5.2.3. Реалізація API-інтерфейсів

Сервер надає набір REST-ендпоінтів для повної автоматизації процесу:

- **/update (POST):** Головний вхідний пункт для ESP32. При отриманні даних сервер не лише оновлює статус місця, а й автоматично створює запис у журналі подій (parking_logs), фіксуючи прибуття (arrival) або від'їзд (departure) автомобіля. Також тут реалізовано скидання броні, якщо машина зайняла місце.
- **/status (GET):** Повертає повний стан системи у форматі JSON: список усіх місць, актуальні метеодані та останні 10 записів з історії.
- **/reserve (POST):** Дозволяє користувачеві віддалено змінити статус is_reserved. Реалізована перевірка: якщо місце вже зайняте фізично (status=1), бронювання стає неможливим.
- **/save-hourly-stat (POST):** Функція для створення «зліпку» стану парковки (статистики), що дозволяє відстежувати динаміку використання протягом доби.

5.2.4. Тестування та верифікація API за допомогою Postman

Для перевірки коректності роботи серверної логіки та взаємодії з базою даних на етапі розробки використовувався інструментарій Postman. Це дозволило протестувати всі ендпоінти сервера без необхідності фізичного підключення мікроконтролера ESP32.

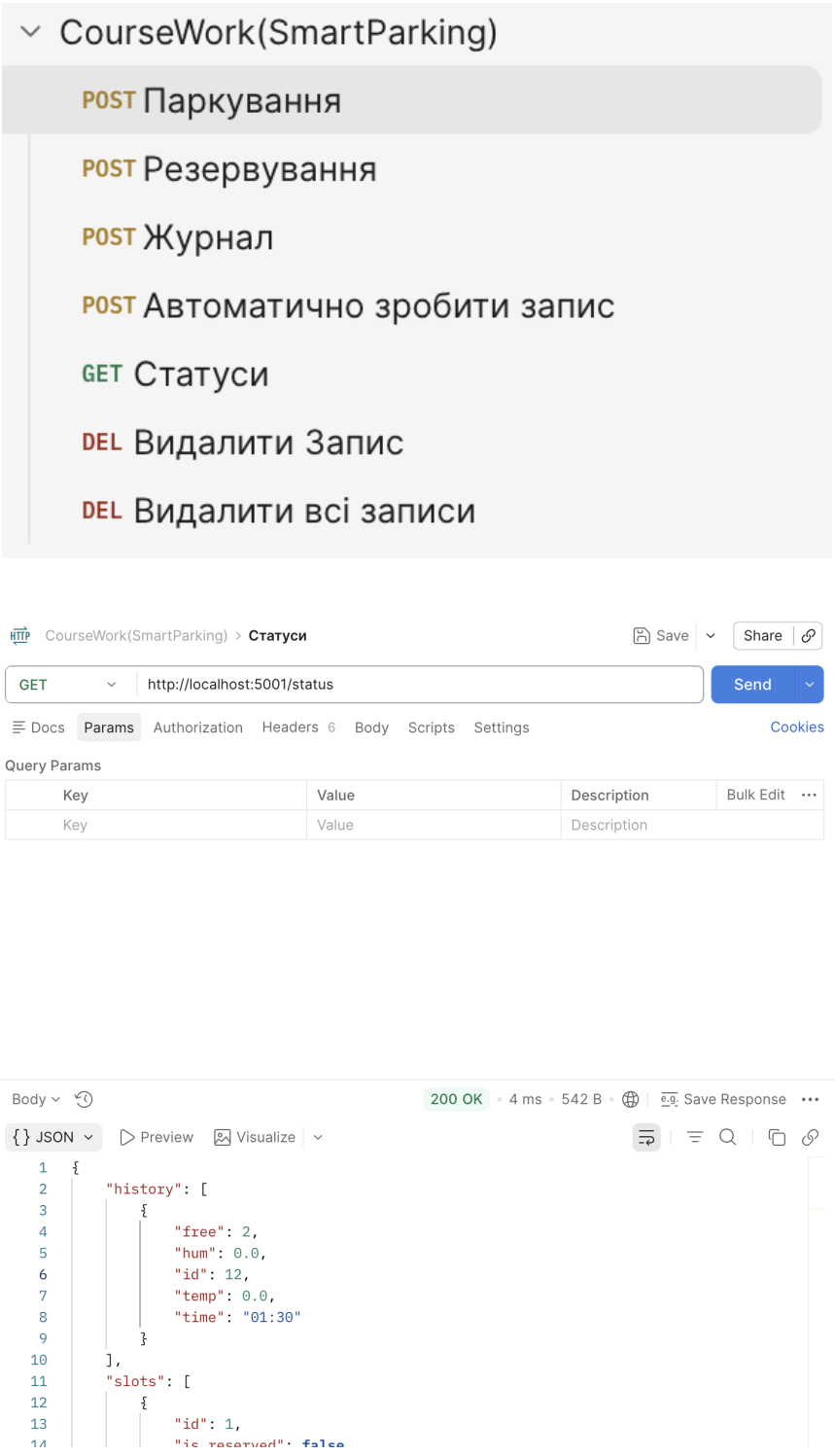


Рис. 5.2 Вікно Postman

5.3. База даних (MySQL)

Для забезпечення цілісності даних, підтримки функцій реального часу та накопичення статистики в проекті використано реляційну базу даних MySQL.

5.3.1. Опис структури та реальних даних таблиць

Згідно з архітектурою проекту, база даних складається з трьох основних таблиць, кожна з яких виконує свою роль у життєвому циклі системи.

1. Таблиця parking_slots (Поточний стан місць)

Таблиця відображає актуальний статус кожного паркувального модуля.

- **id:** Унікальний номер паркомісця.
- **status:** Фізичний стан (0 — вільно, 1 — зайнято).
- **updated_at:** Часова мітка останньої зміни стану, що дозволяє відстежувати актуальність даних.
- **is_reserved:** Логічний прапор (0/1) для системи онлайн-бронювання.

id	status	updated_at	is_reserved
1	0	2026-05-14 03:02:33	0
2	0	2026-05-14 03:03:33	0
NULL	NULL	NULL	NULL

Рис. 5.3.1 Таблиця parking_slots

2. Таблиця parking_logs (Журнал подій)

Ця таблиця автоматично наповнюється сервером при кожній зміні статусу датчиками HC-SR04. Вона пов'язана з таблицею слотів через ідентифікатор slot_id.

- **action:** Тип події (arrival — заїзд автомобіля, departure — виїзд).

- **timestamp:** Точний час події з точністю до секунди.
- **Аналітика:** Як видно з реальних даних таблиці, система чітко фіксує послідовність дій

id	slot_id	action	timestamp
32	1	arrival	2026-05-13 01:20:58
33	1	departure	2026-05-13 01:21:26
34	1	arrival	2026-05-13 01:21:33
35	1	departure	2026-05-13 01:21:54

Рис. 5.3.2 Таблиця parking_logs

3. Таблиця hourly_stats (Статистика та клімат)

Використовується для агрегації загальних показників системи.

- **free_slots:** Загальна кількість доступних місць на момент зняття метрики.
- **temperature / humidity:** Показники з датчика DHT22.
- **Призначення:** Дані з цієї таблиці використовуються для побудови графіків завантаженості та моніторингу умов середовища на веб-панелі.

id	recorded_at	free_slots	temperature	humidity
22	2026-05-14 02:58:04	2	20.2	34.9

Рис. 5.3.2 Таблиця parking_logs

5.3.2. Зв’язки та цілісність даних

Система використовує ідентифікатор **id** з таблиці parking_slots як ключ для зв'язку з іншими таблицями.

1. **Зв'язок 1:N (Один до багатьох):** Один слот може мати сотні записів у parking_logs, що дозволяє аналізувати історію використання кожного конкретного місця.

2. Дані в hourly_stats дозволяють швидко отримувати зріз стану всієї парковки без необхідності складних обчислень по логах, що значно прискорює роботу фронтенд-частини.

5.4. Веб-додаток

Веб-інтерфейс проекту реалізований як динамічний SPA-додаток (Single Page Application), що забезпечує моніторинг парковки в реальному часі без необхідності оновлення сторінки. Основний акцент зроблено на інформативності та миттєвому відгуку системи на зміни в базі даних.

5.4.1. Структура та візуалізація (HTML/CSS)

Інтерфейс має чіткий поділ на зони:

- **Header:** містить логотип та індикатор «LIVE», що сигналізує про активне з'єднання.
- **Weather Widgets:** Картки з актуальною температурою та вологістю, дані для яких беруться безпосередньо з об'єкта data.weather.
- **Parking Grid:** Центральна частина, що складається з карток паркомісць (parking-card). Кожна картка має візуальний елемент (іконку автомобіля) та кнопку керування.

5.4.2. Динамічна логіка (JavaScript та Fetch API)

Логіка клієнтської частини базується на асинхронних запитах до Flask-сервера. Ключові механізми включають:

- **Автоматичне оновлення (Polling):** За допомогою функції setInterval(updateUI, 2000) додаток кожні 2 секунди опитує сервер. Це забезпечує мінімальну затримку між фізичним заїздом авто на місце та зміною кольору на екрані.
- **Двостороння взаємодія:**

- Функція toggleReservation(id) дозволяє користувачу надсилати POST-запити для бронювання місця.
- Функція updateUI динамічно змінює класи CSS (free, occupied, reserved) та стан кнопок (наприклад, блокування кнопки, якщо місце зайняте).

5.4.3. Робота з даними та історією

В інтерфейсі реалізовано повноцінне керування логами та статистикою:

- **Генерація таблиць:** Таблиця stats-table наповнюється динамічно через перебір масиву data.history. Кожен рядок містить кнопку видалення, яка викликає функцію deleteLog(id).
- **Адміністрування:** Додано можливість повного очищення історії за допомогою функції clearHistory(), яка взаємодіє з DELETE-ендпоінтом сервера. Це дозволяє скидати статистику без прямого доступу до бази даних MySQL.

5.4.4. Обробка станів та помилок

Програмний код фронтенду передбачає захист від некоректних дій:

1. **Захист від дублювання:** Якщо слот має статус 1 (зайнято), кнопка бронювання стає неактивною (disabled = true), а текст змінюється на «Місце зайняте».
2. **Валідація видалення:** Операції видалення логів та очищення історії захищені через confirm(), що запобігає випадковому видаленню даних користувачем.

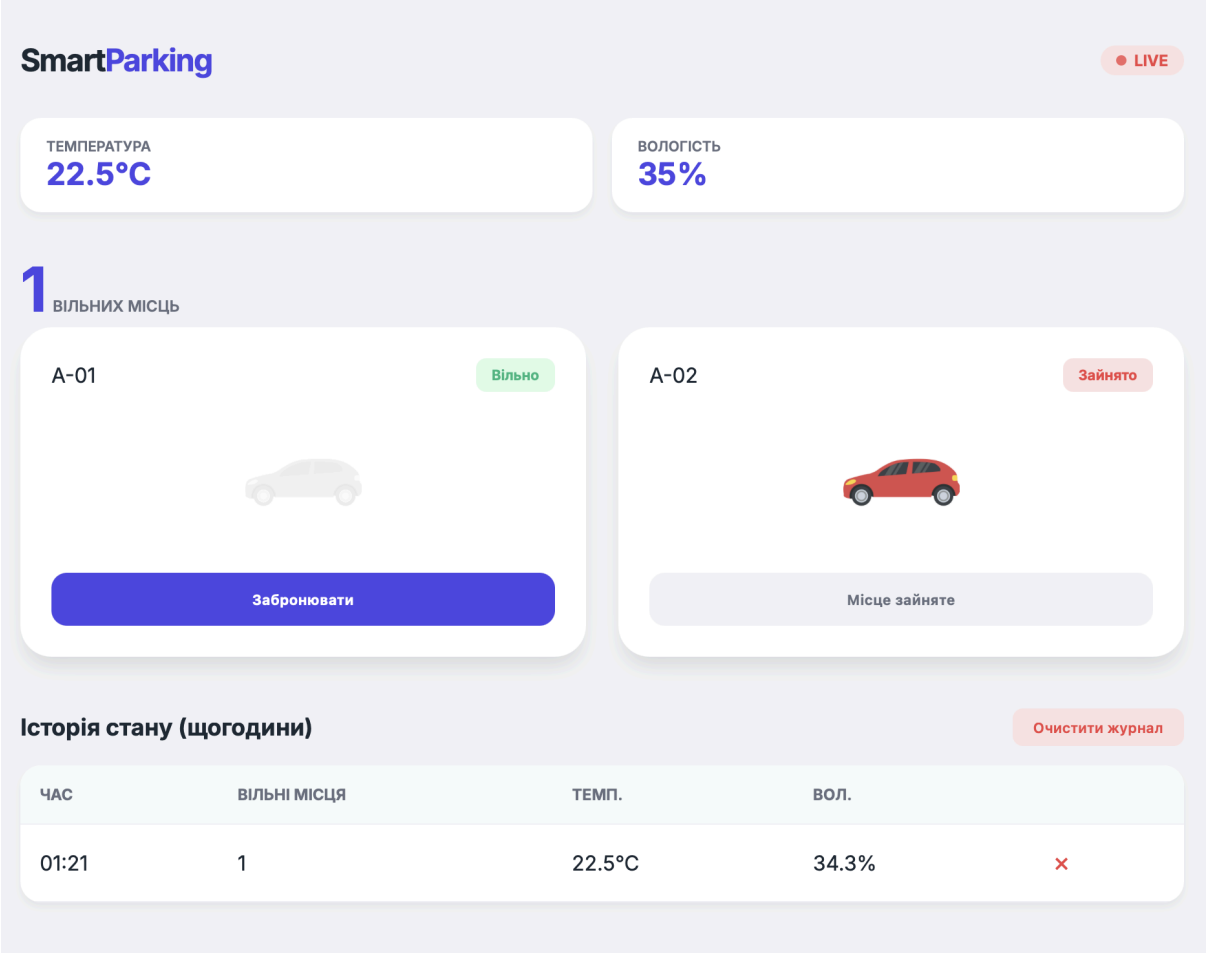
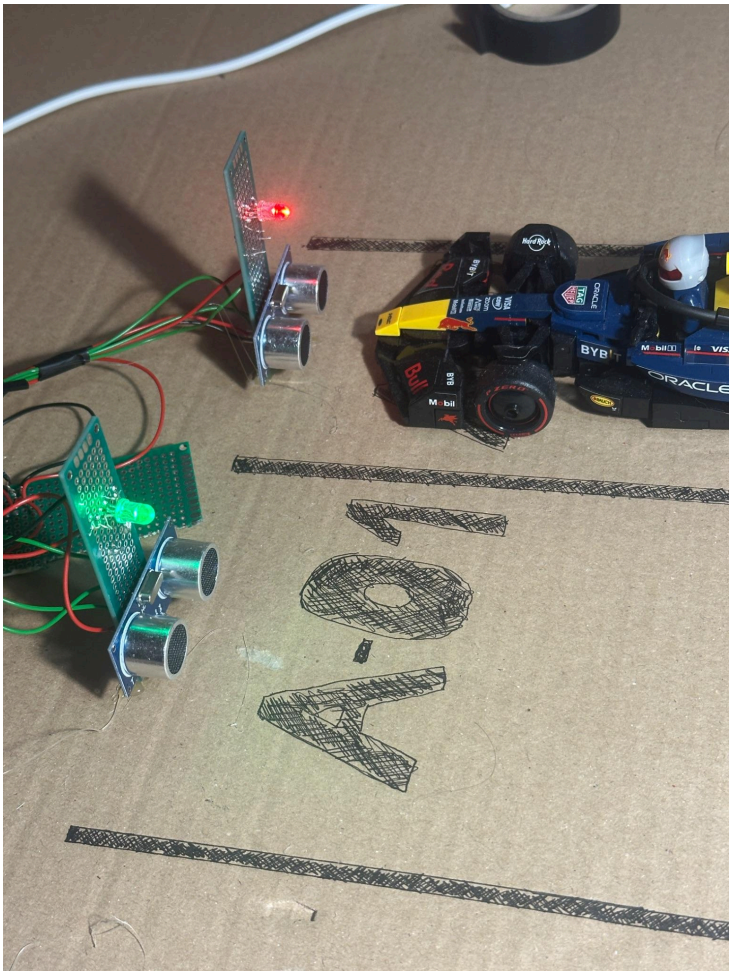


Рис. 5.4 Веб-додаток

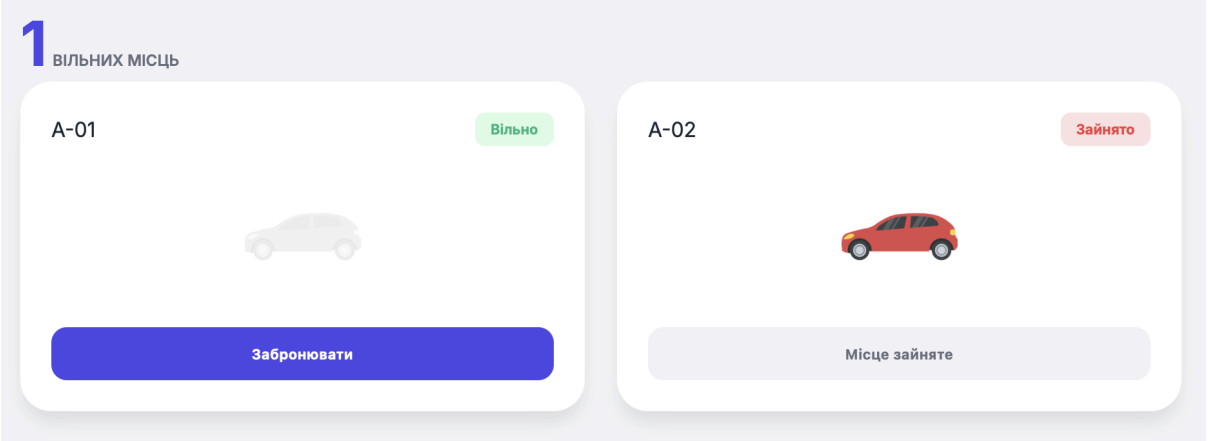
6. ДЕМОНСТРАЦІЯ ПРАЦЕЗДАТНОСТІ ПРОЕКТУ

На завершальному етапі розробки було проведено комплексне тестування системи «Smart Parking», яке підтвердило стабільну взаємодію між апаратними модулями, сервером та веб-інтерфейсом.

Веб-додаток коректно відображає стани паркувальних місць у реальному часі. Нижче наведені скріншоти, що демонструють основні режими роботи:



Машина стоїть на паркомісці А-02



Вигляд блоку з парковкою сайту коли машинка заїхала

id	status	updated_at	is_reserved
1	0	2026-05-14 04:54:54	0
2	1	2026-05-14 04:53:09	0

База Даних коли машинка заїхала

Посилання на GitHub (https://github.com/StasYaremchuk/SmartParking)
Посилання на демонстрацію проекту (https://drive.google.com/drive/folders/1uiRMeIVpWo6HgFP367yupnHj10OGzITN?usp=share_link)

ВИСНОВКИ

У ході виконання курсового проекту було розроблено та успішно реалізовано автоматизовану систему моніторингу паркувального простору «Smart Parking» на базі мікроконтролера ESP32. Результати роботи дозволяють зробити наступні висновки:

1. **Інтеграція апаратних рішень:** Спроектовано та зібрано діючий макет системи, що включає три функціональні модулі: вузол в'їзду з автоматичним шлагбаумом та два модулі моніторингу паркомісць. Використання ультразвукових датчиків HC-SR04 та кліматичного сенсора DHT22 забезпечило високу точність збору даних про стан об'єктів та навколишнє середовище.
2. **Ефективність серверної архітектури:** Розроблено стабільну Back-end частину на фреймворку Flask. Використання Flask-SQLAlchemy дозволило організувати надійну взаємодію з базою даних MySQL, забезпечивши цілісність транзакцій та швидку обробку запитів від апаратної частини через REST API.

3. **Інтерактивність та UX:** Створено сучасний веб-інтерфейс на базі JavaScript, який у реальному часі візуалізує стан парковки. Впровадження механізму «polling» та асинхронних запитів Fetch API дозволило мінімізувати затримку відображення даних до 2 секунд, що є критично важливим для систем реального часу.
4. **Гнучкість керування даними:** Реалізовано повноцінну систему логування подій (arrival, departure) та накопичення статистики. Веб-інтерфейс надає адміністративні можливості для моніторингу, видалення окремих записів або повного очищення журналів, що робить систему автономною та зручною в експлуатації.
5. **Перспективи розвитку:** Розроблена модульна архітектура проекту дозволяє легко масштабувати систему (додавати нові паркомісця) та інтегрувати додаткові функції, такі як розпізнавання номерних знаків за допомогою комп'ютерного зору або впровадження платіжних шлюзів.

Таким чином, розроблена система повністю відповідає поставленим технічним вимогам і демонструє практичне застосування технологій Інтернету речей (IoT) для вирішення актуальних проблем міської інфраструктури.